



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

گزارش آزمایش سوم

اعضای گروه

کیارش شجاعی ارانی

سینا فعالیت

پارسا ملکیان

نام درس

آزمایشگاه طراحی سامانه‌های رقمی

نیم‌سال تابستان ۱۴۰۴-۱۴۰۵

گروه یکشنبه

نام استاد درس

دکتر اجلالی

چکیده: در این آزمایش دو نوع مقایسه‌کننده با زبان توصیف سخت‌افزار Verilog و با استفاده از توصیف جریان داده طراحی و شبیه‌سازی شد. در بخش نخست، ابتدا یک مقایسه‌کننده یک‌بیتی با سه خروجی بزرگ‌تر، مساوی و کوچک‌تر ساخته شد و سپس با اتصال چهار نمونه از این پیمان، یک مقایسه‌کننده چهاربیتی ترکیبی و سلسله‌مراتبی به دست آمد. در بخش دوم، یک مقایسه‌کننده سریال ترتیبی طراحی شد که پس از بازنشانی، بیت‌های دو عدد را از پرارزش‌ترین بیت به کم‌ارزش‌ترین بیت دریافت می‌کند و نتیجه‌ی مقایسه‌ی پیشوند دریافت‌شده را پس از هر نبض ساعت در خروجی نشان می‌دهد. مطابق محدودیت صورت آزمایش، پیمان‌های طراحی فقط از دستور assign برای توصیف منطق استفاده می‌کنند و در مقایسه‌کننده سریال نیز ذخیره‌سازی حالت به کمک بازخورد پیوسته‌ی ساختار ارباب-برده انجام شده است. برای ارزیابی، هر دو مدار با نرم‌افزار Icarus Verilog 12.0 شبیه‌سازی شدند. تمام ۲۵۶ حالت ممکن ورودی چهاربیتی در مدار موازی و تمام ۲۵۶ زوج ورودی در مدار سریال، در هر چهار مرحله‌ی دریافت بیت، بدون خطا آزموده شدند.

واژه‌های کلیدی: وریلاگ، مقایسه‌کننده، توصیف جریان داده، طراحی سلسله‌مراتبی، مدار ترکیبی، مدار ترتیبی، آی‌کاروس وریلاگ

۱ مقدمه

مقایسه‌کننده‌ی دیجیتال مداری است که دو عدد دودویی را دریافت می‌کند و رابطه‌ی بین آن‌ها را به صورت بزرگ‌تر بودن، مساوی بودن یا کوچک‌تر بودن مشخص می‌سازد. این مدار را می‌توان به شکل موازی طراحی کرد تا تمام بیت‌ها را هم‌زمان بررسی کند، یا به صورت سریال ساخت تا در هر نبض ساعت فقط یک جفت بیت را دریافت کرده و نتیجه‌ی مقایسه‌ی انجام‌شده تا آن لحظه را نگه دارد.

هدف این آزمایش، آشنایی عملی با توصیف جریان داده^۱، طراحی سلسله‌مراتبی^۲، تفاوت مدارهای ترکیبی و ترتیبی و همچنین نوشتن آزمون خودکار برای یک مدار دیجیتال است. تمام معادلات منطقی مدارهای اصلی با دستور assign نوشته شده‌اند و صحت عملکرد آن‌ها با شبیه‌ساز Icarus Verilog بررسی شده است.

^۱Dataflow description
^۲Hierarchical design

۱-۱ تعریف مسئله

این آزمایش شامل دو بخش است:

- طراحی یک مقایسه‌کننده‌ی یک‌بیتی به روش جریان داده و استفاده از چهار نمونه‌ی آن برای ساخت یک مقایسه‌کننده‌ی چهاربیتی ترکیبی و سلسله‌مراتبی.
- طراحی یک مقایسه‌کننده‌ی سریال ترتیبی در قالب تنها یک پیمان، به گونه‌ای که پیمان در ابتدای کار بازنشانی شود، سپس بیت‌های ورودی را در هر نبض ساعت دریافت کند و نتیجه‌ی مقایسه‌ی پیشوند دریافت‌شده را در خروجی نمایش دهد.
- پیاده‌سازی تمام منطق پیمان‌های اصلی فقط با توصیف جریان داده و بدون استفاده از ساختارهای رفتاری مانند `initial`، `always` و `case`.

۲-۱ اهداف آزمایش

اهداف اصلی انجام آزمایش عبارت‌اند از:

- استخراج معادلات بولی یک مقایسه‌کننده‌ی یک‌بیتی؛
- استفاده‌ی مجدد از یک پیمان‌ی ساده در یک طراحی بزرگ‌تر؛
- رعایت اولویت بیت پرارزش‌تر در مقایسه‌ی اعداد چندبیتی؛
- طراحی حالت برای مقایسه‌ی سریال و حفظ نتیجه‌ی قطعی‌شده؛
- درک نقش بازنشانی و ساعت در مدار ترتیبی؛
- آزمون جامع مدارها با تولید خودکار تمام ورودی‌های ممکن.

۲ مفاهیم اولیه

۱-۲ توصیف جریان داده

در توصیف جریان داده، ارتباط میان ورودی و خروجی به وسیله‌ی عبارت‌های منطقی و دستور `assign` بیان می‌شود. سمت راست یک انتساب پیوسته هر زمان که یکی از ورودی‌های آن تغییر

کند دوباره ارزیابی می‌شود و نتیجه روی سیم خروجی قرار می‌گیرد. برای نمونه، عبارت زیر ترکیب دروازه‌های AND و NOT را توصیف می‌کند:

```
10 assign gt = a & ~b;
```

این روش برای بیان مدارهای ترکیبی بسیار مناسب است، زیرا خروجی مدار ترکیبی فقط به مقدار فعلی ورودی‌ها وابسته است.

۲-۲ مدار ترکیبی و مدار ترتیبی

در مدار ترکیبی هیچ حافظه‌ای وجود ندارد و خروجی فقط تابع ورودی جاری است. مقایسه‌کننده‌ی چهاربیتی این آزمایش یک مدار ترکیبی است. در مقابل، مدار ترتیبی علاوه بر ورودی جاری، به نتیجه یا حالت قبلی نیز وابسته است. مقایسه‌کننده‌ی سریال باید به یاد داشته باشد که در بیت‌های قبلی کدام عدد بزرگ‌تر بوده است؛ بنابراین یک مدار ترتیبی محسوب می‌شود.

۳-۲ اصل اولویت بیت پرارزش‌تر

در مقایسه‌ی دو عدد دودویی، نخستین بیت متفاوت از سمت چپ نتیجه را تعیین می‌کند. برای مثال در مقایسه‌ی 1010_2 و 1001_2 ، دو بیت نخست برابرند، اما در بیت بعدی مقدار عدد اول برابر ۱ و مقدار عدد دوم برابر ۰ است. در نتیجه عدد اول بزرگ‌تر است و بیت پایانی دیگر اثری بر نتیجه ندارد.

۳ طراحی مقایسه‌کننده‌ی چهاربیتی

۱-۳ مقایسه‌کننده‌ی یک‌بیتی

ورودی‌های پیمانه‌ی یک‌بیتی با a و b و خروجی‌های آن با gt ، eq و lt نمایش داده می‌شوند. جدول درستی مدار در جدول ۱ آمده است.

با توجه به جدول درستی، معادلات بولی سه خروجی به صورت زیر به دست می‌آیند:

جدول ۱: جدول درستی مقایسه کننده ی یک بیتی

lt	eq	gt	b	a
۰	۱	۰	۰	۰
۱	۰	۰	۱	۰
۰	۰	۱	۰	۱
۰	۱	۰	۱	۱

$$gt = a \wedge \neg b \quad (۱)$$

$$eq = \neg(a \oplus b) \quad (۲)$$

$$lt = \neg a \wedge b \quad (۳)$$

این معادلات در پرونده ی comparator_1bit.v مستقیماً در خطوط ۱۰ تا ۱۲ با سه دستور assign پیاده سازی شده اند:

```
10 assign gt = a & ~b;
11 assign eq = ~(a ^ b);
12 assign lt = ~a & b;
```

۲-۳ ساختار سلسله مراتبی مدار چهاربیتی

پیمانه ی comparator_4bit.v شامل چهار نمونه از مقایسه کننده ی یک بیتی است. نمونه ی c3 پرارزش ترین بیت و نمونه ی c0 کم ارزش ترین بیت را مقایسه می کند. خطوط ۱۴ تا ۲۸ این پیمانه، چهار نمونه و منطق ترکیب خروجی ها را نشان می دهند. خروجی های هر نمونه در سه بردار میانی bit_eq، bit_gt و bit_lt ذخیره می شوند.

```

14 comparator_1bit c3(a[3], b[3], bit_gt[3], bit_eq[3], bit_lt[3]);
15 comparator_1bit c2(a[2], b[2], bit_gt[2], bit_eq[2], bit_lt[2]);
16 comparator_1bit c1(a[1], b[1], bit_gt[1], bit_eq[1], bit_lt[1]);
17 comparator_1bit c0(a[0], b[0], bit_gt[0], bit_eq[0], bit_lt[0]);
18
19 assign gt = bit_gt[3]
20           | (bit_eq[3] & bit_gt[2])
21           | (bit_eq[3] & bit_eq[2] & bit_gt[1])
22           | (bit_eq[3] & bit_eq[2] & bit_eq[1] & bit_gt[0]);
23 assign eq = &bit_eq;
24 assign lt = bit_lt[3]
25           | (bit_eq[3] & bit_lt[2])
26           | (bit_eq[3] & bit_eq[2] & bit_lt[1])
27           | (bit_eq[3] & bit_eq[2] & bit_eq[1] & bit_lt[0]);
28 endmodule

```

شکل ۱: کد و ساختار سلسله‌مراتبی را نشان می‌دهد.

```

1 `timescale 1ns/1ps
2
3 module comparator_1bit (
4     input wire a,
5     input wire b,
6     output wire gt,
7     output wire eq,
8     output wire lt
9 );
10     assign gt = a & ~b;
11     assign eq = ~(a ^ b);
12     assign lt = ~a & b;
13 endmodule
14
15 `timescale 1ns/1ps
16
17 module comparator_4bit (
18     input wire [3:0] a,
19     input wire [3:0] b,
20     output wire gt,
21     output wire eq,
22     output wire lt
23 );
24     wire [3:0] bit_gt;
25     wire [3:0] bit_eq;
26     wire [3:0] bit_lt;
27
28     comparator_1bit c3(a[3], b[3], bit_gt[3], bit_eq[3], bit_lt[3]);
29     comparator_1bit c2(a[2], b[2], bit_gt[2], bit_eq[2], bit_lt[2]);
30     comparator_1bit c1(a[1], b[1], bit_gt[1], bit_eq[1], bit_lt[1]);
31     comparator_1bit c0(a[0], b[0], bit_gt[0], bit_eq[0], bit_lt[0]);
32
33     assign gt = bit_gt[3]
34             | (bit_eq[3] & bit_gt[2])
35             | (bit_eq[3] & bit_eq[2] & bit_gt[1])
36             | (bit_eq[3] & bit_eq[2] & bit_eq[1] & bit_gt[0]);
37
38     assign eq = &bit_eq;
39
40     assign lt = bit_lt[3]
41             | (bit_eq[3] & bit_lt[2])
42             | (bit_eq[3] & bit_eq[2] & bit_lt[1])
43             | (bit_eq[3] & bit_eq[2] & bit_eq[1] & bit_lt[0]);
44 endmodule

```

شکل ۱: پیاده‌سازی جریان داده و ساختار سلسله‌مراتبی مقایسه‌کننده‌ی چهاربیتی

۳-۳ استخراج خروجی‌های چهاربیتی

برای ساده‌سازی، نتیجه‌ی مقایسه‌ی هر بیت با e_i, g_i و l_i نمایش داده می‌شود. چون بیت شماره‌ی ۳ پرارزش‌ترین بیت است، معادله‌ی بزرگ‌تر بودن به صورت زیر است:

$$GT = g_3 \vee (e_3 \wedge g_2) \vee (e_3 \wedge e_2 \wedge g_1) \vee (e_3 \wedge e_2 \wedge e_1 \wedge g_0) \quad (4)$$

به همین ترتیب، معادله‌ی کوچک‌تر بودن برابر است با:

$$LT = l_3 \vee (e_3 \wedge l_2) \vee (e_3 \wedge e_2 \wedge l_1) \vee (e_3 \wedge e_2 \wedge e_1 \wedge l_0) \quad (5)$$

دو عدد فقط زمانی برابرند که هر چهار جفت بیت با هم برابر باشند:

$$EQ = e_3 \wedge e_2 \wedge e_1 \wedge e_0 \quad (6)$$

همان‌طور که در خط ۲۳ کد بالا دیده می‌شود، عملگر کاهش AND شرط برابری را به صورت کوتاه پیاده‌سازی می‌کند. بدین ترتیب خروجی تنها تابع ورودی فعلی است و هیچ مسیر ذخیره‌سازی در مدار چهاربیتی وجود ندارد.

۴ طراحی مقایسه‌کننده‌ی سریال

۱-۴ روش دریافت بیت‌ها

در این طراحی فرض شده است که بیت‌های دو عدد از پرارزش‌ترین بیت به کم‌ارزش‌ترین بیت وارد می‌شوند. پیش از شروع مقایسه، ورودی reset یک می‌شود تا حالت مدار به «برابر تا اینجا» بازگردد. پس از غیرفعال شدن بازنشانی، در هر چرخه‌ی ساعت یک بیت روی ورودی a و یک بیت روی ورودی b قرار می‌گیرد. نتیجه پس از لبه‌ی بالارونده‌ی ساعت در سه خروجی ظاهر می‌شود.

۲-۴ کدگذاری حالت

برای نگه‌داری نتیجه‌ی مقایسه‌ی انجام‌شده، یک حالت دوبیتی در نظر گرفته شده است. جدول حالت‌ها در جدول ۲ آمده است.

جدول ۲: کدگذاری حالت مقایسه‌کننده‌ی سریال

حالت	مفهوم	gt	eq	lt
00	برابر تا این مرحله	۰	۱	۰
10	عدد a بزرگ‌تر است	۱	۰	۰
01	عدد a کوچک‌تر است	۰	۰	۱
11	حالت استفاده‌نشده	—	—	—

حالت 00 برای بازنشانی انتخاب شده است. تا زمانی که دو بیت ورودی برابر باشند، حالت تغییر نمی‌کند. اگر در حالت برابری $a = ۱$ و $b = ۰$ شود، مدار وارد حالت 10 می‌شود. اگر $a = ۰$ و $b = ۱$ شود، مدار وارد حالت 01 می‌شود. پس از ورود به یکی از دو حالت بزرگ‌تر یا کوچک‌تر، بیت‌های بعدی نتیجه را تغییر نمی‌دهند؛ زیرا آن‌ها کم‌ارزش‌ترند.

۳-۴ منطق حالت بعدی

اگر s_1 نشان‌دهنده‌ی بزرگ‌تر بودن و s_0 نشان‌دهنده‌ی کوچک‌تر بودن باشد، منطق حالت بعدی از روابط زیر پیروی می‌کند:

$$s_1^+ = s_1 \vee (\neg s_1 \wedge \neg s_0 \wedge a \wedge \neg b) \quad (۷)$$

$$s_0^+ = s_0 \vee (\neg s_1 \wedge \neg s_0 \wedge \neg a \wedge b) \quad (۸)$$

وجود s_1 و s_0 در ابتدای عبارت‌های OR باعث می‌شود نتیجه‌ی قطعی شده حفظ شود. شرط $\neg s_1 \wedge \neg s_0$ نیز اجازه می‌دهد ورودی جدید فقط در حالت برابری روی نتیجه اثر بگذارد.

۴-۴ ذخیره‌ی حالت با توصیف جریان داده

در توصیف متداول و قابل سنتز وریلاگ، حافظه‌ی لبه‌ای با یک بلوک always نوشته می‌شود؛ اما صورت آزمایش استفاده از هر توصیف دیگری را ممنوع کرده است. به همین دلیل، حالت با دو بخش ارباب و برده و بازخورد انتساب پیوسته مدل شده است. خطوط ۱۶ تا ۲۴ پرونده‌ی serial_comparator.v منطق حالت بعدی، ذخیره‌سازی و رمزگشایی خروجی را نشان می‌دهند:

```
16 assign next_state[1] = state[1] | (~state[1] & ~state[0] & a & ~b);
17 assign next_state[0] = state[0] | (~state[1] & ~state[0] & ~a & b);
18
19 assign master = reset ? 2'b00 : (~clk ? next_state : master);
20 assign state = reset ? 2'b00 : ( clk ? master : state);
21
22 assign gt = state[1];
23 assign eq = ~state[1] & ~state[0];
24 assign lt = state[0];
```

عملکرد این دو انتساب به ترتیب زیر است:

۱. هنگامی که reset=1 است، هر دو بخش مقدار 00 می‌گیرند.
۲. هنگامی که clk=0 است، بخش ارباب مقدار حالت بعدی را دنبال می‌کند و بخش برده مقدار قبلی را نگه می‌دارد.
۳. هنگامی که clk=1 می‌شود، بخش ارباب مقدار خود را نگه می‌دارد و بخش برده مقدار ذخیره‌شده در ارباب را دریافت می‌کند.
۴. در نتیجه، خروجی قابل مشاهده در لبه‌ی بالارونده‌ی ساعت به‌روزرسانی می‌شود.

این روش محدودیت «فقط assign» آزمایش را در شبیه‌ساز Icarus Verilog برآورده می‌کند. با این حال، بازخورد انتساب پیوسته روش متعارف برای سنتز مدار واقعی نیست و اگر محدودیت آزمایش وجود نداشت، استفاده از یک فلیپ‌فلاپ لبه‌ای با توصیف ترتیبی انتخاب استانداردتری بود.

شکل ۲ برای درج تصویر کامل پیمانه‌ی سریال در نظر گرفته شده است.

```

`timescale 1ns/1ps

module serial_comparator (
    input wire clk,
    input wire reset,
    input wire a,
    input wire b,
    output wire gt,
    output wire eq,
    output wire lt
);
    wire [1:0] state;
    wire [1:0] master;
    wire [1:0] next_state;

    assign next_state[1] = state[1] | (~state[1] & ~state[0] & a & ~b);
    assign next_state[0] = state[0] | (~state[1] & ~state[0] & ~a & b);

    assign master = reset ? 2'b00 : (~clk ? next_state : master);
    assign state = reset ? 2'b00 : ( clk ? master : state);

    assign gt = state[1];
    assign eq = ~state[1] & ~state[0];
    assign lt = state[0];
endmodule

```

شکل ۲: پیاده‌سازی تک‌پیمانه‌ای و جریان داده‌ی مقایسه‌کننده‌ی سریال

۵-۴ مثال مرحله‌به‌مرحله

برای نمایش عملکرد مدار سریال، دو عدد $a = 1010_2$ و $b = 1001_2$ در نظر گرفته می‌شوند. جدول ۳ نتیجه‌ی مدار را پس از هر نبض ساعت نشان می‌دهد.

جدول ۳: مقایسه‌ی سریال 1010_2 و 1001_2 از بیت پرارزش‌تر

مرحله	بیت‌های ورودی a, b	پیشوند مقایسه‌شده	نتیجه
بازنشانی	—	بدون بیت	مساوی
نبض ۱	۱, ۱	۱ و ۱	مساوی
نبض ۲	۰, ۰	۱۰ و ۱۰	مساوی
نبض ۳	۱, ۰	۱۰۰ و ۱۰۱	بزرگ‌تر
نبض ۴	۰, ۱	۱۰۰۱ و ۱۰۱۰	بزرگ‌تر

۵ پیاده‌سازی و آزمون

۱-۵ پرونده‌های پروژه

پرونده‌های استفاده‌شده در این آزمایش عبارت‌اند از:

- comparator_1bit.v: پیمانه‌ی یک‌بیتی با سه انتساب پیوسته؛
- comparator_4bit.v: پیمانه‌ی چهاربیتی شامل چهار نمونه‌ی یک‌بیتی؛
- serial_comparator.v: مقایسه‌کننده‌ی سریال تک‌پیمانه‌ای؛
- tb_comparator_4bit.v: آزمون جامع مقایسه‌کننده‌ی چهاربیتی؛
- tb_serial_comparator.v: آزمون جامع مقایسه‌کننده‌ی سریال.

ساختارهای رفتاری initial، حلقه و task فقط در آزمون‌گرها استفاده شده‌اند و جزئی از سخت‌افزار طراحی شده نیستند. سه پرونده‌ی اصلی مدار شامل هیچ case، initial، always، تابع یا وظیفه‌ای نیستند.

۲-۵ محیط اجرا

شبیه‌سازی با نسخه‌ی ۰/۱۲ نرم‌افزار Icarus Verilog نصب‌شده در مسیر سیستمی /usr/bin انجام شد. گزینه‌ی -Wall برای نمایش هشدارها و گزینه‌ی -g2012 برای انتخاب زبان وریلاگ/سیستم‌وریلاگ نسخه‌ی ۲۰۱۲ به کار رفت.

۳-۵ آزمون مقایسه‌کننده‌ی چهاربیتی

دو حلقه‌ی تو در تو تمام مقادیر صفر تا ۱۵ را برای هر یک از ورودی‌های a و b تولید می‌کنند. بنابراین تعداد کل مقایسه‌ها برابر است با:

$$16 \times 16 = 256 \quad (9)$$

پس از اعمال هر زوج ورودی، خروجی‌های مدار با نتایج عملگرهای مرجع <، = و > مقایسه می‌شوند. در صورت مشاهده اختلاف، آزمون‌گر ورودی و خروجی اشتباه را چاپ و شبیه‌سازی را متوقف می‌کند. در اجرای انجام‌شده هیچ اختلافی مشاهده نشد.

```

kshyst .../Desktop /University/DSDL3 15:01 71%
parator_1bit.v comparator_4bit.v tb_comparator_4bit.v
vvp .build/comparator4.out next_state[1:0] 00
PASS: all 256 four-bit comparisons reset
tb_comparator_4bit.v:31: $finish called at 256000 (1ps)
  
```

شکل ۳: کامپایل و عبور موفق تمام ۲۵۶ حالت مقایسه‌کننده‌ی چهاربیتی

۴-۵ آزمون مقایسه‌کننده‌ی سریال

در آزمون سریال نیز تمام ۲۵۶ زوج عدد چهاربیتی بررسی می‌شوند. پیش از ارسال هر زوج، وظیفه‌ی do_reset حالت مدار و خروجی 010 را کنترل می‌کند. سپس حلقه‌ی داخلی از بیت شماره‌ی ۳ تا بیت شماره‌ی صفر حرکت می‌کند. پس از هر لبه‌ی بالارونده‌ی ساعت، خروجی واقعی با نتیجه‌ی پیشوندی که تا همان لحظه دریافت شده است مقایسه می‌شود. بنابراین علاوه بر ۲۵۶ کنترل بازنشانی، تعداد بررسی‌های پیشوندی برابر است با:

$$1024 = 256 \times 4 \quad (10)$$

این روش حالت برابری، تفاوت در هر یک از چهار موقعیت و حفظ نتیجه پس از مشاهده‌ی نخستین بیت متفاوت را پوشش می‌دهد. تمام بررسی‌ها با موفقیت عبور کردند.

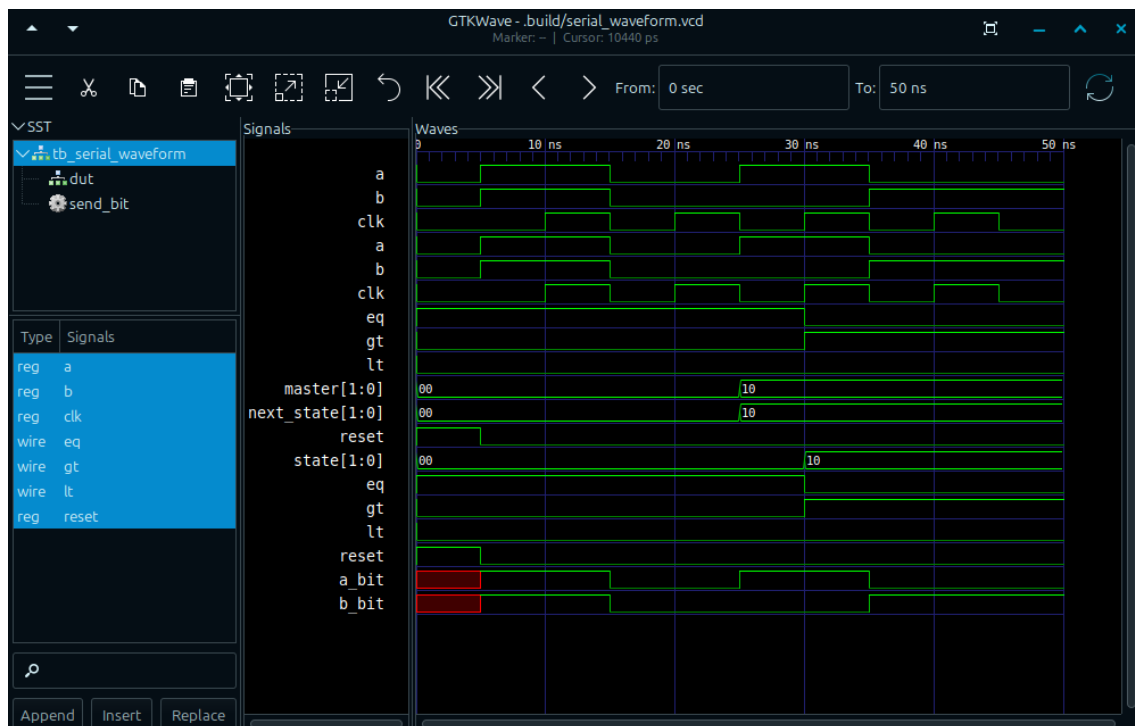
```

kshyst .../Desktop /University/DSDL3 15:01 71%
_comparator.v tb_serial_comparator.v
vvp .build/serial.out reset
PASS: all 256 serial comparisons at every prefix a bit
tb_serial_comparator.v:83: $finish called at 3584000 (1ps)
  
```

شکل ۴: کامپایل و عبور موفق تمام حالت‌ها و پیشوندهای مقایسه‌کننده‌ی سریال

۵-۵ تصویر شکل موج

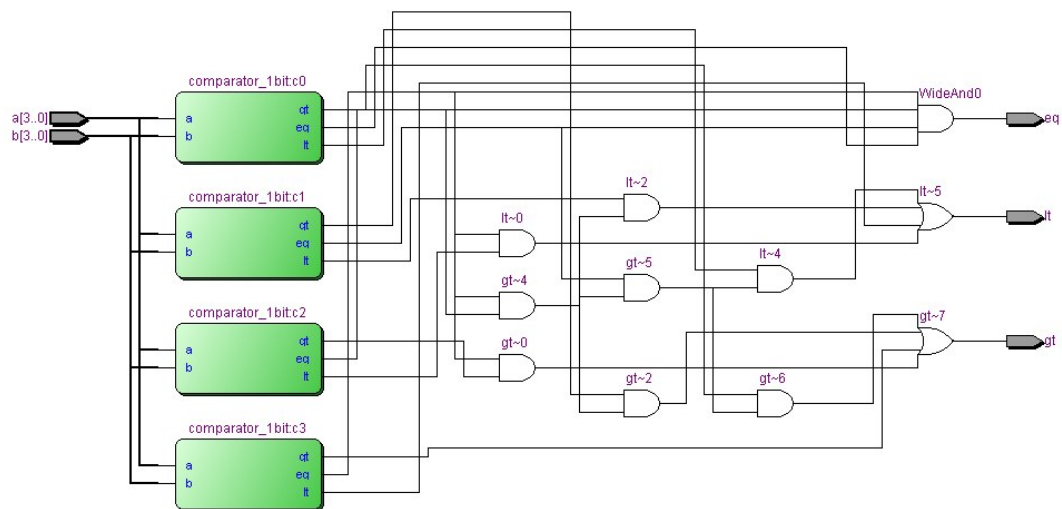
برای نمایش دیداری رفتار ترتیبی، یک اجرای کوتاه از مثال ۱۰۱۰۲ و ۱۰۰۱۲ در نمایش دهنده‌ی شکل موج انجام شود. تصویر باید سیگنال‌های `clk`، `reset`، `a`، `b`، `gt`، `eq` و `lt` را نشان دهد. در ابتدای تصویر، `eq` باید پس از بازنشانی یک باشد. پس از دو جفت بیت برابر نیز مقدار آن باید حفظ شود. در نبض سوم `gt` یک می‌شود و با وجود متفاوت بودن بیت چهارم، مقدار بزرگ‌تر بودن باید ثابت بماند.



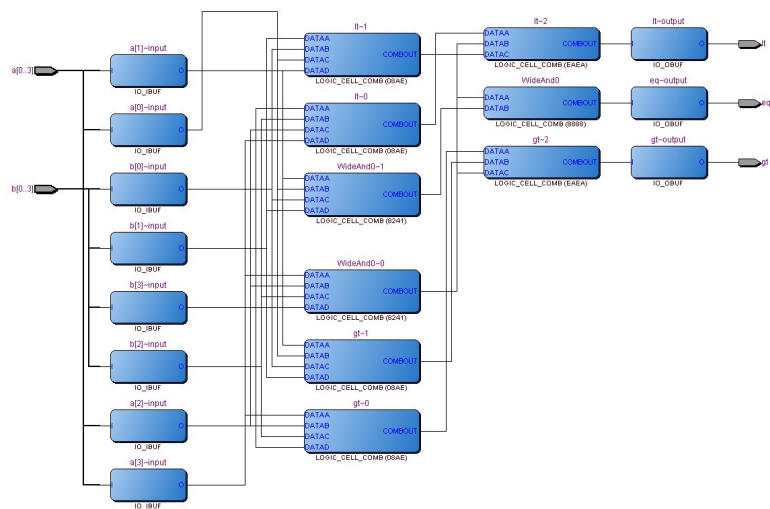
شکل ۵: شکل موج بازنشانی و مقایسه‌ی سریال مرحله‌به‌مرحله

۶ خروجی‌های کوارتوس

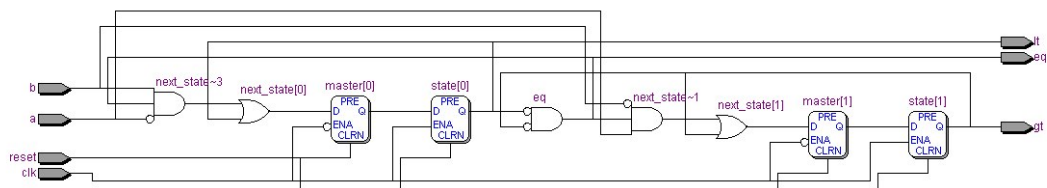
این آزمایش به طور خاص چون ویندوز نصب نداشتم و نتونستم کوارتوس نصب کنم، هم‌تیمی بنده آقای فعالیت زحمت سنتز را کشیدند و خروجی را به من دادند.



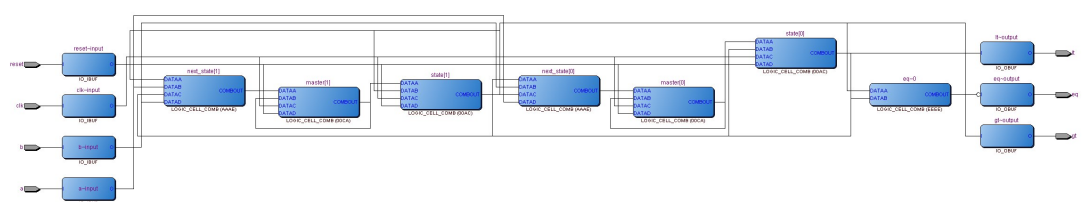
شکل ۶: سنتر شده مقایسه کننده ۴ بیتی



شکل ۷: نگارش پورت های مقایسه کننده ۴ بیتی



شکل ۸: سنتز شده مقایسه کننده سریال



شکل ۹: نگارش پورت های مقایسه کننده سریال

۷ دستورهای اجرا

ابتدا در پوشه ی پروژه، پوشه ی خروجی ساخته می شود:

```
mkdir -p .build
```

دستورهای کامپایل و اجرای آزمون مقایسه کننده ی چهاربیتی عبارت اند از:

```
iverilog -g2012 -Wall -s tb_comparator_4bit \
-o .build/comparator4.out \
comparator_1bit.v comparator_4bit.v tb_comparator_4bit.v

vvp .build/comparator4.out
```

خروجی مورد انتظار برابر است با:

```
PASS: all 256 four-bit comparisons
```

دستورهای کامپایل و اجرای آزمون مقایسه‌کننده‌ی سریال عبارت‌اند از:

```
iverilog -g2012 -Wall -s tb_serial_comparator \  
-o .build/serial.out \  
serial_comparator.v tb_serial_comparator.v  
  
vvp .build/serial.out
```

خروجی مورد انتظار برابر است با:

```
PASS: all 256 serial comparisons at every prefix
```

۸ نتایج آزمایش

نتایج نهایی در جدول ۴ خلاصه شده‌اند.

جدول ۴: خلاصه‌ی نتایج شبیه‌سازی

نتیجه	تعداد بررسی	نوع طراحی	مدار
موفق	۲۵۶ زوج ورودی	ترکیبی و سلسله‌مراتبی	مقایسه‌کننده‌ی چهاربیتی
موفق	۱۰۲۴ پیشوند و ۲۵۶ بازنشانی	ترتیبی و تک‌پیمانه‌ای	مقایسه‌کننده‌ی سریال

کامپایل هر دو طراحی با گزینه‌ی نمایش هشدارها انجام شد و پس از افزودن مقیاس زمانی یکسان به پیمانه‌ها، هیچ هشدار کامپایل باقی نماند. آزمون جامع نشان داد که خروجی‌های eq، gt و lt در تمام حالت‌ها یک‌داغ^۳ هستند؛ یعنی در هر لحظه دقیقاً یکی از آن‌ها مقدار یک دارد.

One-hot^۳

۹ جمع‌بندی

در بخش نخست آزمایش، معادلات ساده‌ی یک مقایسه‌کننده‌ی یک‌بیتی به کمک انتساب پیوسته پیاده‌سازی شدند. سپس چهار نمونه از این پیمان‌ها با رعایت اولویت بیت پرارزش‌تر به یکدیگر متصل شدند و یک مقایسه‌کننده‌ی چهاربیتی کاملاً ترکیبی و سلسله‌مراتبی تشکیل دادند. در بخش دوم، نتیجه‌ی مقایسه در یک حالت دوییتی ذخیره شد تا مدار بتواند بیت‌ها را به صورت سریال دریافت کند. حالت اولیه برابری است و نخستین جفت بیت متفاوت از سمت پرارزش‌تر نتیجه‌ی نهایی را تعیین می‌کند.

محدودیت استفاده‌ی انحصاری از assign باعث شد ذخیره‌سازی سریال با بازخورد پیوسته‌ی ارباب-برده مدل شود. این ساختار در شبیه‌ساز هدف آزمایش به درستی کار کرد، هرچند برای یک طراحی قابل‌سنتز صنعتی، استفاده از توصیف ترتیبی لبه‌ای روش متعارف‌تری است. در نهایت، آزمون جامع تمام ورودی‌های ممکن صحت هر دو مدار و نتیجه‌ی هر مرحله از مقایسه‌ی سریال را تأیید کرد.